



MAY 3, 2026

INFORMATION SECURITY LAB
ASSIGNMENT NO 4

MUHAMMAD.BILAL
FA24-BSE-023
TO:AMBREEN GUL



Task 1

Code Screenshots

```
task2.py U task_3.py U Activity_1.java U CSP_colors U Assignment_3_Task_1.py U Assignment_4_task_1.py U X
SE-4-VScode > IS Lab > Assignment_4_task_1.py > encrypt
1  # Simple RSA implementation (for learning only)
2  # Step 1: Choose two small prime numbers
3  p = 11
4  q = 13
5  # Step 2: Calculate n = p * q
6  n = p * q
7  # Step 3: Calculate Euler's Totient function
8  phi = (p - 1) * (q - 1)
9  # Step 4: Choose e (public key exponent)
10 e = 7 # must be coprime with phi
11 # Step 5: Find d (private key exponent)
12 # d is modular inverse of e mod phi
13 def mod_inverse(e, phi):
14     for d in range(1, phi):
15         if (e * d) % phi == 1:
16             return d
17 d = mod_inverse(e, phi)
18 print("Public Key (e, n):", (e, n))
19 print("Private Key (d, n):", (d, n))
20 # ----- ENCRYPTION FUNCTION -----
21 def encrypt(message, e, n):
22     cipher = []
23     for char in message:
24         # Convert character to number (ASCII)
25         m = ord(char)
26         # RSA formula: c = m^e mod n
27         c = pow(m, e, n)
28         cipher.append(c)
29     return cipher
```

```
task2.py U task_3.py U Activity_1.java U CSP_colors U Assignment_3_Task_1.py U Assignment_4_task_1.py U X
SE-4-VScode > IS Lab > Assignment_4_task_1.py > encrypt
30 # ----- DECRYPTION FUNCTION -----
31 def decrypt(cipher, d, n):
32     message = ""
33     for c in cipher:
34         # RSA formula: m = c^d mod n
35         m = pow(c, d, n)
36         message += chr(m)
37     return message
38 # ----- TEST -----
39 text = "ALI" # your name or short string
40 print("\nOriginal Message:", text)
41 encrypted = encrypt(text, e, n)
42 print("Encrypted:", encrypted)
43 decrypted = decrypt(encrypted, d, n)
44 print("Decrypted:", decrypted)
```

Code Overview

This program demonstrates the basic working of the RSA encryption algorithm using small numbers so the mathematical logic is easy to understand. First, two small prime numbers p and q are chosen. These primes are used to calculate n , which is simply their product and becomes part of both the public and private keys. Then, we compute Euler's Totient function $\phi = (p-1)(q-1)$, which is important for determining the relationship between the public and private keys. After that, we select a number e such that it is coprime with ϕ (meaning they share no common factors except 1). This e becomes the public key exponent.

Mod Inverse Function

To find the private key, we create the `mod_inverse()` function. This function calculates d , which is the modular inverse of e with respect to ϕ . In simple terms, d is chosen such that $(e \times d) \bmod \phi = 1$. We create this function because finding the modular inverse is a key mathematical step in RSA, and separating it into a function keeps the code clean and reusable instead of repeating the logic.

Encrypt Function

The `encrypt()` function is used to convert a message into encrypted form using the public key (e, n). Inside this function, each character of the message is first converted into its ASCII number using `ord()`. Then the RSA formula $c = m^e \bmod n$ is applied to generate encrypted values. We store all encrypted values in a list. This function is created to clearly separate the encryption logic and allow easy reuse for different messages.

Decrypt Function

The `decrypt()` function reverses the process using the private key (d, n). It takes each encrypted number and applies the formula $m = c^d \bmod n$ to recover the original ASCII value, which is then converted back into a character using `chr()`. This function is necessary to demonstrate how the private key is used to retrieve the original message securely.

Finally, the program tests RSA by encrypting and decrypting a short string (such as a name). The output shows that the encrypted message looks like a list of numbers, and after decryption, the original text is recovered correctly. This demonstrates the core idea of RSA: even though the encryption process is public, only the private key can correctly decrypt the message.

Task 2

Code Screenshots

```
Assignment_3_Task_1.py U Assignment_4_task_1.py U Assignment_3_Task_2.py U Assignment_4_Task_2.py U X
SE-4-VScode > IS Lab > Assignment_4_Task_2.py > ...
1  import hashlib
2  # ----- RSA KEY GENERATION -----
3  p = 11
4  q = 13
5  n = p * q
6  phi = (p - 1) * (q - 1)
7  e = 7 # public exponent
8  # Find modular inverse
9  def mod_inverse(e, phi):
10     for d in range(1, phi):
11         if (e * d) % phi == 1:
12             return d
13  d = mod_inverse(e, phi)
14  print("Public Key (e, n):", (e, n))
15  print("Private Key (d, n):", (d, n))
16  # ----- HASH FUNCTION -----
17  def hash_message(message):
18     # SHA256 hashing
19     hash_obj = hashlib.sha256(message.encode())
20     return int(hash_obj.hexdigest(), 16)
21  # ----- SIGN FUNCTION -----
22  def sign(message, d, n):
23     h = hash_message(message)
24     signature = pow(h, d, n) # sign using private key
25     return signature
26  # ----- VERIFY FUNCTION -----
27  def verify(message, signature, e, n):
28     h = hash_message(message)
29     h_from_signature = pow(signature, e, n) # decrypt signature
```

```
Assignment_3_Task_1.py U Assignment_4_task_1.py U Assignment_3_Task_2.py U Assignment_4_Task_2.py U X
SE-4-VScode > IS Lab > Assignment_4_Task_2.py > ...
27  def verify(message, signature, e, n):
30     # Compare both hashes
31     return h % n == h_from_signature
32  # ----- TEST -----
33  message = "Hello"
34  print("\nOriginal Message:", message)
35  signature = sign(message, d, n)
36  print("Signature:", signature)
37  # Verify correct message
38  is_valid = verify(message, signature, e, n)
39  print("Verification (Original):", is_valid)
40  # ----- MODIFY MESSAGE -----
41  fake_message = "Hella" # slight change
42  print("\nModified Message:", fake_message)
43  is_valid_fake = verify(fake_message, signature, e, n)
44  print("Verification (Modified):", is_valid_fake)
```

Code Overview

This program demonstrates how a digital signature works using the RSA algorithm and hashing. First, RSA keys are generated by selecting two small prime numbers p and q , then calculating $n = p \times q$ and Euler's Totient $\phi = (p-1)(q-1)$. A public key exponent e is chosen, and the private key d is computed using the `mod_inverse()` function. This function is created to calculate the modular inverse of e , which is necessary to form the private key. It is separated into its own function to keep the code organized and reusable, since this calculation is a core part of RSA.

Hash Message Function

Next, the `hash_message()` function is used to convert the message into a fixed-size hash using SHA256. Instead of signing the entire message directly, we sign its hash because hashing is faster and ensures a fixed-length representation. This function is created to isolate the hashing process, making it easy to reuse and modify if needed. The `sign()` function creates the digital signature. It first computes the hash of the message, then encrypts that hash using the private key with the formula $\text{signature} = \text{hash}^d \bmod n$. This function is necessary to clearly represent the signing process and to separate it from other parts of the program.

Verify Function

The `verify()` function checks whether a signature is valid. It hashes the received message again and also decrypts the signature using the public key with the formula $\text{hash_from_signature} = \text{signature}^e \bmod n$. If both hashes match, the signature is valid. This function is created to encapsulate the verification logic and demonstrate how anyone with the public key can verify authenticity without needing the private key.

Finally, the program tests the system by signing an original message and verifying it successfully. Then, the message is slightly modified, and verification is attempted again. This time it fails, showing that even a small change in the message produces a completely different hash, which breaks the signature. This demonstrates the key purpose of digital signatures: ensuring both authenticity and data integrity.

Code Output 1

```
PS D:\Comsats Semesters\Comsats semester 4\Information Security\Lab\Assignment NO 4> & 'C:\Users\dell\AppData\Local\Microsoft\WindowsApps\python3.
s\dell\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '54040' '--' 'd:\Comsats Semesters\Comsats semester 4
urity\Lab\Assignment NO 4\Assignment_4_task_1.py'
Public Key (e, n): (7, 143)
Private Key (d, n): (103, 143)

Original Message: ALI
Encrypted: [65, 54, 83]
Decrypted: ALI
```

Code Output 2

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS D:\Comsats Semesters\Comsats semester 4\Information Security\Lab\Assignment NO 4> & 'C:\Users\dell\AppData
s\dell\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '58689' '--' 'd
urity\Lab\Assignment NO 4\Assignment_4_Task_2.py'
Public Key (e, n): (7, 143)
Private Key (d, n): (103, 143)

Original Message: Hello
Signature: 30
Verification (Original): True

Modified Message: Hella
Verification (Modified): False
PS D:\Comsats Semesters\Comsats semester 4\Information Security\Lab\Assignment NO 4>
```

The END